

Classificação Automática da Dificuldade de Questões de Programação Utilizando Aprendizado de Máquina e LLMs

Carlos Eduardo Pereira Oliveira - 116233

carlos.oliveira1@ufv.br

Universidade Federal de Viçosa
Viçosa, Brasil

Luis Felipe Martins Pereira - 112710

luis.f.pereira@ufv.br

Universidade Federal de Viçosa
Viçosa, Brasil

ABSTRACT

Este trabalho investiga se o nível de dificuldade de uma questão de programação pode ser inferido automaticamente apenas a partir de seu enunciado textual. A tarefa foi modelada como classificação supervisionada em cinco níveis (de muito fácil a muito difícil), derivados das avaliações empíricas de dificuldade atribuídas por alunos a 80 questões da disciplina INF110 da Universidade Federal de Viçosa. Comparamos quatro famílias de modelos clássicos de aprendizado de máquina — Regressão Logística, K-Nearest Neighbors (KNN), Support Vector Machines (SVM) e Random Forest — sobre representações TF-IDF, utilizando validação cruzada estratificada, ajuste de hiperparâmetros e prevenção explícita de vazamento de informação, tendo como piso de comparação um *baseline* de classe majoritária. O melhor modelo (KNN) atingiu F1-macro de 0,30 na escala de cinco níveis, contra 0,13 do *baseline*. Uma análise complementar de granularidade mostrou que, ao reduzir a escala para três níveis, o F1-macro sobe para 0,44, evidenciando o forte impacto do número de classes e do desbalanceamento. Adicionalmente, integramos um Grande Modelo de Linguagem (LLM, o Google Gemini) em três papéis: *baseline* de classificação, gerador de explicações e extrator de características semânticas. Concluímos que o enunciado textual carrega sinal real, porém parcial, para a tarefa.

KEYWORDS

aprendizado de máquina, classificação de texto, processamento de linguagem natural, TF-IDF, LLMs, dificuldade de questões de programação

1 INTRODUÇÃO

A classificação automática da dificuldade de questões de programação é um problema relevante em contextos educacionais, como plataformas de ensino e sistemas de recomendação de exercícios. A capacidade de estimar automaticamente o nível de dificuldade pode auxiliar tanto alunos quanto professores na organização do processo de aprendizagem, na montagem de listas balanceadas e na personalização de trilhas de estudo.

A principal pergunta que orientou o trabalho foi: *é possível inferir automaticamente o nível de dificuldade de uma questão de programação apenas a partir de seu enunciado textual?* Para respondê-la, tratamos a tarefa como um problema de classificação supervisionada e a estudamos sobre uma base real de avaliações de dificuldade feitas por alunos.

As principais contribuições deste trabalho são:

- um *pipeline* reproduzível e modular que vai da ingestão dos dados brutos à avaliação dos modelos, com configuração centralizada e separação por fonte de dados;
- uma comparação criteriosa de quatro famílias de modelos clássicos sobre representações TF-IDF, com validação cruzada estratificada, busca de hiperparâmetros, prevenção explícita de vazamento de informação e comparação contra um *baseline* trivial;
- uma análise de granularidade que contrasta a tarefa em cinco e em três níveis, quantificando o impacto do número de classes;
- a integração de um LLM (Google Gemini) em três papéis estratégicos — *baseline* de classificação, explicador e extrator de características semânticas; e
- um recomendador de exercícios baseado em conteúdo, capaz de combinar múltiplas fontes de questões.

2 TRABALHOS RELACIONADOS

A representação de textos por TF-IDF (*Term Frequency–Inverse Document Frequency*) é uma técnica consolidada em recuperação de informação e classificação de texto [7], por ponderar termos de acordo com sua frequência local e raridade no corpus. Sobre essas representações, modelos clássicos de aprendizado de máquina são amplamente empregados: a classificação por vizinhos mais próximos [4], as máquinas de vetores de suporte [3] e as florestas aleatórias [2] figuram entre as abordagens mais robustas para dados esparsos, ao lado de métodos lineares. Toda a implementação apoiou-se em bibliotecas maduras do ecossistema Python, em especial o *scikit-learn* [6] e o NLTK [1].

Mais recentemente, Grandes Modelos de Linguagem (LLMs) como o Gemini [5] têm sido aplicados a tarefas de compreensão textual em cenários de poucos exemplos (*few-shot*), tornando-se candidatos naturais tanto como *baselines* fortes quanto como ferramentas de extração de conhecimento. A estimativa automática de dificuldade, especificamente, é um problema desafiador porque a dificuldade percebida depende não só do texto, mas de conhecimentos prévios e habilidades do solucionador, o que motiva o uso combinado de representações clássicas e semânticas.

3 MATERIAIS E MÉTODOS

3.1 Base de Dados

A base principal foi fornecida pelo professor e corresponde a questões da disciplina INF110 da Universidade Federal de Viçosa. Cada questão possui um enunciado textual e, quando avaliada, um conjunto de notas de dificuldade atribuídas empiricamente por alunos em uma

Table 1: Distribuição das 80 questões rotuladas do INF110 pelos cinco níveis de dificuldade.

Nível	# Questões
Muito fácil	7
Fácil	31
Médio	32
Difícil	9
Muito difícil	1
Total	80

escala de 1 (muito fácil) a 5 (muito difícil). Ao todo, são 138 enunciados; destes, 80 receberam avaliações de alunos, totalizando 2.244 avaliações coletadas ao longo de três anos (308 em 2024, 1.233 em 2025 e 703 em 2026). Os enunciados têm, em média, 236 palavras (mediana de 220, variando de 67 a 610).

Para obter um rótulo único por questão, agregamos as notas dos alunos pela **média aritmética arredondada ao inteiro mais próximo**, mapeando cada inteiro de 1 a 5 a um dos cinco níveis canônicos. Optou-se pela média, e não pela moda, para evitar classificações enganosas em casos bimodais — por exemplo, quando metade dos alunos considera a questão muito fácil e a outra metade muito difícil, a média a posiciona corretamente como “médio”. A Tabela 1 apresenta a distribuição resultante.

Observa-se um **desbalanceamento severo**: as classes centrais (fácil e médio) concentram 63 das 80 questões, enquanto “muito difícil” possui um único exemplo. Esse desequilíbrio, inerente à percepção real dos discentes, é um fator determinante para os resultados e é discutido na Seção 4. As 58 questões sem avaliação não entram no treinamento supervisionado, mas são aproveitadas pelo recomendador (Seção 3.7).

3.2 Decisão de Modelagem: Cinco Níveis

Diferentemente da proposta inicial, que previa três níveis (fácil, médio e difícil), a versão final adota **cinco níveis**, em correspondência direta com a escala de 1 a 5 utilizada pelos alunos. Essa decisão preserva toda a granularidade da percepção dos discentes e evita o descarte de informação que ocorreria ao colapsar a escala. O custo dessa escolha é tornar a tarefa mais difícil e acentuar o desbalanceamento nas extremidades, o que avaliamos explicitamente na análise de granularidade (Seção 4.2).

3.3 Pré-processamento e Representação

Cada enunciado passou por uma rotina de limpeza padronizada: conversão para minúsculas, remoção de trechos de código, remoção de acentuação e de pontuação, colapso de espaços e remoção de *stop-words* em português e inglês (via NLTK [1]). Os enunciados limpos foram convertidos em vetores TF-IDF [7], considerando unigramas e bigramas, com no máximo 5.000 atributos e descartando termos que aparecessem em menos de dois documentos. A vetorização foi encapsulada em um *pipeline* de modo a ser reajustada de forma independente em cada partição de treino (Seção 3.5). O uso de *embeddings* semânticos densos foi considerado e permanece como trabalho futuro (Seção 5).

3.4 Modelos

Foram exploradas quatro famílias de modelos, escolhidas por representarem paradigmas distintos de aprendizado e por permitirem comparação justa em um mesmo cenário: **Regressão Logística** (linear), **KNN** [4] (baseado em vizinhança, com distância do cosseno), **SVM linear** [3] (margem máxima) e **Random Forest** [2] (*ensemble* de árvores). Para mitigar o desbalanceamento, os modelos que o suportam foram treinados com ponderação de classes inversamente proporcional à sua frequência. Os hiperparâmetros de cada família foram selecionados por busca em grade (*grid search*) sob a mesma validação cruzada, otimizando o F1-macro. Como piso de comparação, incluímos um *baseline* trivial que sempre prevê a classe majoritária.

3.5 Protocolo de Avaliação

Como a base rotulada é pequena (80 exemplos), um único particionamento treino/teste produziria métricas instáveis e pouco confiáveis. Adotamos, portanto, **validação cruzada estratificada** com 5 partições, reportando média e desvio-padrão entre elas. Quando alguma classe é rara demais para ser estratificada (caso de “muito difícil”, com um exemplo), o procedimento recai automaticamente para uma validação cruzada não estratificada, com aviso.

Dois cuidados metodológicos foram centrais. Primeiro, a **prevenção de vazamento de informação**: o vetorizador TF-IDF é reajustado *dentro* de cada partição de treino, nunca tendo acesso aos documentos de teste — evitando que estatísticas globais do corpus contaminem a avaliação. Segundo, o uso de **predições out-of-fold** para construir o relatório por classe e a matriz de confusão, de modo que cada questão seja avaliada exatamente pelo modelo que não a viu durante o treino. As métricas reportadas são acurácia, F1-macro (que trata todas as classes igualmente, penalizando o mau desempenho nas classes raras) e F1-ponderado. Todos os experimentos usaram semente aleatória fixa para garantir reprodutibilidade.

3.6 Integração de LLM

Um LLM proprietário de ponta (Google Gemini [5]) foi integrado ao sistema em três papéis estratégicos:

- (1) **Baseline de classificação**: o modelo classifica diretamente a dificuldade do enunciado, em modo *zero-shot* ou *few-shot*, servindo de referência para os modelos clássicos;
- (2) **Explicador**: gera justificativas em linguagem natural para a dificuldade atribuída, citando os conceitos e algoritmos envolvidos, dando transparência às predições; e
- (3) **Extrator de características semânticas**: identifica os conceitos necessários para resolver cada questão (por exemplo, recursão, programação dinâmica, grafos e estruturas de dados), gerando atributos que podem ser combinados às *features* TF-IDF.

O arcabouço de avaliação comparativa — modelos clássicos *versus* LLM *versus* modelos clássicos enriquecidos com as características extraídas pelo LLM — está implementado e é reprodutível. Nesta etapa, a avaliação quantitativa em larga escala concentra-se nos modelos clássicos; a comparação empírica completa com o LLM, que depende de chamadas à API e do respectivo custo, constitui a continuação imediata do trabalho.

Table 2: Desempenho na escala de cinco níveis, sob validação cruzada (5 partições), ordenado por F1-macro. Valores: média $\pm$ desvio-padrão entre partições.

Modelo	Acurácia	F1-macro	F1-pond.
KNN ($k=5$)	0,425 $\pm$ 0,083	0,304 $\pm$ 0,104	0,397
Reg. Logística	0,362 $\pm$ 0,073	0,261 $\pm$ 0,047	0,348
SVM linear	0,425 $\pm$ 0,061	0,260 $\pm$ 0,066	0,383
Random Forest	0,425 $\pm$ 0,073	0,244 $\pm$ 0,048	0,384
<i>Baseline</i>	0,350	0,133	0,185

3.7 Recomendador de Exercícios

Como componente adicional, implementamos um recomendador baseado em conteúdo: cada questão é representada por seu vetor TF-IDF, o perfil do aluno é modelado como o centroide das questões que já resolveu e recomendam-se as questões não resolvidas mais similares (similaridade do cosseno), com filtro opcional pelo próximo nível de dificuldade. Por não depender de rótulos, o recomendador combina múltiplas fontes num só catálogo, incluindo — além do INF110 — questões do Neps Academy (1.448), do SPOJ (301) e da OBI (353), totalizando mais de 2.200 exercícios candidatos.

4 RESULTADOS E DISCUSSÃO

4.1 Classificação em Cinco Níveis

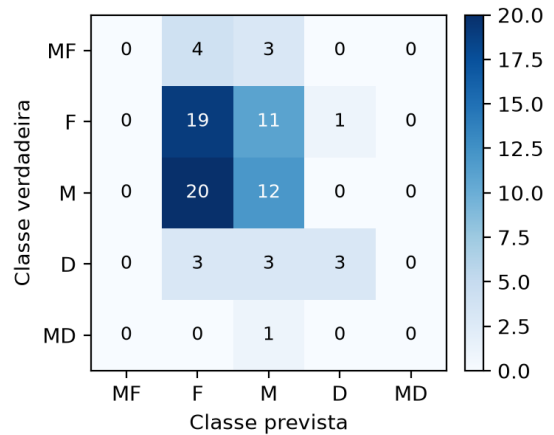
A Tabela 2 apresenta o desempenho das quatro famílias sob validação cruzada. O KNN obteve o melhor F1-macro (0,30), seguido de perto, em acurácia, por SVM e Random Forest. Todos os modelos superam com folga o *baseline* de classe majoritária, especialmente em F1-macro (0,30 contra 0,13): embora o ganho em acurácia sobre o *baseline* seja modesto, o F1-macro muito superior indica que os modelos efetivamente distribuem predições entre as classes centrais, em vez de colapsar em uma só.

A Figura 1 traz a matriz de confusão do melhor modelo. Dois padrões se destacam. Primeiro, as **classes extremas nunca são previstas**: nenhuma questão foi classificada como “muito fácil” ou “muito difícil”, o que é consequência direta de haver apenas 7 e 1 exemplos dessas classes, respectivamente — quantidade insuficiente para que qualquer modelo aprenda suas fronteiras. Segundo, a confusão concentra-se entre **fácil e médio**, classes adjacentes na escala e majoritárias na base, o que é coerente com a natureza ordinal e subjetiva do rótulo.

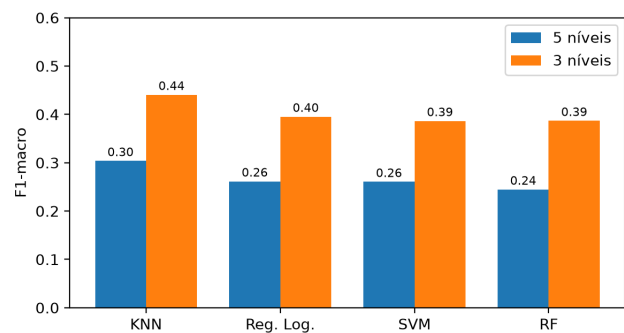
4.2 Análise de Granularidade: Cinco versus Três Níveis

Para quantificar o impacto da granularidade, repetimos todo o protocolo colapsando a escala em três níveis (unindo muito fácil + fácil e difícil + muito difícil), o que também atenua o desbalanceamento. A Tabela 3 e a Figura 2 resumem os resultados.

O F1-macro do melhor modelo sobe de 0,30 (cinco níveis) para 0,44 (três níveis) — um ganho de cerca de 45% —, e a melhora é consistente em todas as famílias. Mais importante, na escala de três níveis a classe “difícil” passa a ser efetivamente prevista (com F1 em torno de 0,48 no KNN), ao contrário do que ocorria nas extremidades da escala de cinco níveis. Isso confirma que boa parte

**Figure 1: Matriz de confusão do melhor modelo (KNN), com predições *out-of-fold*, na escala de cinco níveis. MF = muito fácil, F = fácil, M = médio, D = difícil, MD = muito difícil.****Table 3: Desempenho na escala reduzida de três níveis, sob o mesmo protocolo, ordenado por F1-macro.**

Modelo	Acurácia	F1-macro	F1-pond.
KNN ($k=3$)	0,450 $\pm$ 0,100	0,440 $\pm$ 0,117	0,440
Reg. Logística	0,512 $\pm$ 0,145	0,395 $\pm$ 0,167	0,482
Random Forest	0,562 $\pm$ 0,068	0,387 $\pm$ 0,057	0,514
SVM linear	0,512 $\pm$ 0,121	0,386 $\pm$ 0,142	0,470
<i>Baseline</i>	0,475	0,214	0,307

**Figure 2: F1-macro por modelo nas escalas de cinco e três níveis. A redução de granularidade melhora consistentemente o desempenho.**

da dificuldade da tarefa não está no modelo, e sim na granularidade e no desbalanceamento dos rótulos.

4.3 Resposta à Pergunta de Pesquisa

Os resultados indicam uma resposta *parcialmente afirmativa*: existe sinal aproveitável no enunciado textual — os modelos superam

consistentemente o acaso e o *baseline* trivial em F1-macro —, mas esse sinal é insuficiente para uma classificação confiável em cinco níveis com a base disponível. O fator limitante predominante não parece ser a escolha do modelo (as quatro famílias têm desempenho próximo), e sim a **quantidade e o balanceamento dos dados rotulados**. A dificuldade percebida também depende de fatores externos ao texto (repertório prévio do aluno, familiaridade com paradigmas), o que estabelece um teto natural para abordagens puramente textuais.

5 CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho desenvolveu e avaliou um sistema reprodutível para classificação automática da dificuldade de questões de programação a partir do enunciado, em cinco níveis derivados da percepção real de alunos. Comparamos quatro famílias de modelos clássicos sobre representações TF-IDF, com um protocolo de avaliação cuidadoso (validação cruzada estratificada, sem vazamento de informação, com *baseline* de referência), e integramos um LLM em três papéis. O melhor modelo (KNN) atingiu F1-macro de 0,30 em cinco níveis e 0,44 em três, deixando claro tanto que há sinal textual explorável quanto que a tarefa é fortemente limitada pelo tamanho e pelo desbalanceamento da base.

Como trabalhos futuros, destacam-se: (i) a incorporação de *embeddings* semânticos densos (próprios de LLMs ou de modelos multilíngues abertos) como representação alternativa ou complementar ao TF-IDF; (ii) a execução da comparação quantitativa completa com

as abordagens baseadas em LLM, já suportada pelo arcabouço implementado; (iii) técnicas de mitigação de desbalanceamento, como reamostragem, ou a reformulação da tarefa como classificação ordinal; e (iv) a ampliação e o balanceamento da base de questões rotuladas, idealmente com avaliações de múltiplas instituições, para elevar o teto de desempenho alcançável.

ACKNOWLEDGMENTS

Os autores agradecem ao professor da disciplina INF420 (Inteligência Artificial) da Universidade Federal de Viçosa pela orientação e pela disponibilização da base de dados de avaliações de dificuldade utilizada neste trabalho.

REFERENCES

- [1] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media, Sebastopol, CA.
- [2] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.
- [3] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning* 20, 3 (1995), 273–297.
- [4] Thomas M. Cover and Peter E. Hart. 1967. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27.
- [5] Gemini Team, Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2023).
- [6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [7] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513–523.